

PROMPT ITERATION LOG: Video Editing Platform Value Proposition

Assignment: Prompting Fundamentals on Real Tasks v2

Track: General AI Fluency

Project: Professional Video Editing Platform (Ubuntu-focused)

TASK DESCRIPTION

FL-01 Target Task: Write a value proposition (2–3 paragraphs) for a browser-based video editing platform built for Ubuntu users. The platform solves the dual-boot problem—editing video on Ubuntu without needing an NVIDIA GPU or switching to Windows.

Why this task: This is a real task from my FL-01 audit. I need compelling copy to explain what I'm building and why it matters. The copy will live on the landing page of my portfolio.

BASELINE: Version 0 (Naive Prompt)

The Prompt

| "Write a value proposition for a video editing platform."

The Output

"Our video editing platform is a powerful, intuitive solution for content creators of all levels. With advanced features and a user-friendly interface, you can create professional-quality videos quickly and easily. Whether you're a beginner or an expert, our platform provides everything you need to bring your creative vision to life. Start editing today and see the difference."

Iteration Notes

Note	Response
Technique applied	None — baseline.
What changed in the output	N/A — baseline.
What improved	N/A — baseline is generic and unusable.
What still failed	Everything. Sounds like any SaaS product. No specificity, no audience, no problem statement.

VERSION 1: Role Assignment

Technique Applied

Role Assignment: Assign Claude a specific role or persona to guide its response. This provides context for appropriate tone, knowledge level, and approach.

The Prompt

"You are a technical copywriter who specializes in writing for developer tools and open-source software. Your audience is technical—they use Linux, understand hardware constraints, and value practicality over hype. Write a value proposition for a video editing platform."

The Output

"A browser-based video editor that runs on Ubuntu. No NVIDIA GPU required. No dual-booting. Just open Chrome and edit. Built for developers and creators who refuse to compromise their workflow for proprietary lock-in. Timeline editing, cuts, trimming, MP4 export—all in your browser, all on your machine."

Iteration Notes

Note	Response
Technique applied	Role assignment.
What changed in the output	Tone shifted from generic marketing to technical and direct. Mentioned Ubuntu, NVIDIA, and dual-booting specifically.
What improved	The output now addresses a technical audience. The "no dual-booting" hook is strong and specific.
What still failed	Still vague about what the product actually does. No mention of how it works (WebAssembly, local storage). No credibility.

VERSION 2: Context and Motivation

Technique Applied

Context and Motivation: Provide the "why" behind the product. Explain the problem the user faces and why this solution exists.

The Prompt

"You are a technical copywriter for developer tools. Write a value proposition for a browser-based video editing platform built for Ubuntu users."

Context: The founder is a full-stack developer who uses Ubuntu. They tried DaVinci Resolve but it required an NVIDIA GPU (they have Intel integrated graphics). Other Linux tools crashed or were too basic. They built this to solve their own problem—editing video without rebooting into Windows.

The platform uses WebAssembly for performance and stores footage locally (no uploads). Features: timeline editing, cuts, trimming, MP4 export."

The Output

"I built this because I was tired of rebooting into Windows every time a client sent a rush edit. DaVinci Resolve wouldn't run on my Intel GPU. Kdenlive crashed on export. So I built a browser-based editor that actually works on Ubuntu.

*

It uses WebAssembly for near-native performance. Everything stays local—no uploads, no privacy concerns. Timeline editing, cuts, trimming, MP4 export. No effects. No color grading. Just the essentials, done reliably. Because you shouldn't need an NVIDIA GPU to pay your bills."

Iteration Notes

Note	Response
Technique applied	Context and motivation.
What changed in the output	Added a personal story ("I built this because...") and explained the specific technical choices (WebAssembly, local storage).
What improved	The output now has a human voice and a clear "why." The reader understands the problem and why this solution exists.
What still failed	No examples of what "good" looks like. No demonstration of the workflow. Still abstract.

VERSION 3: Few-Shot Examples

Technique Applied

Few-Shot Examples: Provide demonstrations to enable in-context learning. The examples serve as conditioning for the model to understand the task format and expected output style.

The Prompt

"You are a technical copywriter for developer tools. Write a value proposition for a browser-based video editing platform built for Ubuntu users.

Context: The founder is a full-stack developer who uses Ubuntu and built this to solve the dual-boot problem. Uses WebAssembly and local storage. Features: timeline editing, cuts, trimming, MP4 export.

Here are examples of good value propositions for technical products:

Example 1: 'Postman is an API platform for building and using APIs. Postman simplifies each step of the API lifecycle and streamlines collaboration so you can create better APIs—faster.'

Example 2: 'Figma is a collaborative design tool. It lives in the browser, so your team can design together in real time. No more emailing files back and forth.'

Now write a value proposition for this video editing platform in a similar style—clear, specific, and benefit-focused."

The Output

"A browser-based video editor that lives where you work—in Chrome on Ubuntu. No installation. No NVIDIA GPU. No dual-booting. Just open a tab and edit.

*

Timeline editing, cuts, trimming, and MP4 export—all powered by WebAssembly for native-like performance. Your footage never leaves your machine (IndexedDB storage). No uploads, no privacy risks, no waiting.

*

Built by a developer who was tired of rebooting into Windows for every client edit. Now you can turn around a cut in minutes, not hours."

Iteration Notes

Note	Response
Technique applied	Few-shot examples.
What changed in the output	Structure became cleaner and more benefit-focused. The "lives where you work" framing is stronger. Added "minutes, not hours" as a clear benefit.
What improved	The output now follows a proven pattern (clear, specific, benefit-driven). It feels more polished and credible.
What still failed	No structure or formatting guidance. The output is a wall of text—hard to scan.

VERSION 4: Output Structure

Technique Applied

Output Structure: Specify the desired format to guide response structure. This makes the output more scannable and usable.

The Prompt

"You are a technical copywriter for developer tools. Write a value proposition for a browser-based video editing platform built for Ubuntu users.

Context: Built by a developer to solve the dual-boot problem. Uses WebAssembly and local storage. Features: timeline editing, cuts, trimming, MP4 export.

Use this structure:

- **Headline (one line):** The core promise
- **The Problem (2-3 sentences):** What frustrates the user
- **The Solution (3-4 sentences):** What the product does and how it works
- **The Result (1-2 sentences):** What the user gets

Follow the style of the examples provided."

The Output

Headline: Edit video on Ubuntu. No NVIDIA. No rebooting.

The Problem: Every time a client sends a rush edit, you face the same choice: reboot into Windows and waste 20 minutes, or struggle with Linux tools that crash on export. Your Intel GPU can't run DaVinci Resolve. Your deadline can't wait.

The Solution: A browser-based video editor that runs in Chrome on Ubuntu. WebAssembly delivers near-native performance. IndexedDB keeps everything local—no uploads, no privacy risks. Timeline editing, cuts, trimming, and MP4 export. No effects. No color grading. Just the essentials, done reliably.

The Result: Turn around client cuts in minutes, not hours. Stay in Ubuntu. Ship work. Get paid.

Iteration Notes

Note	Response
Technique applied	Output structure.
What changed in the output	The output now has clear sections with headers. It's scannable and professional.
What improved	The structure makes the value proposition easy to read and digest. Each section has a clear purpose.
What still failed	No quality criteria. No guidance on what "good" looks like for tone, length, or voice.

VERSION 5: Step Decomposition

Technique Applied

Step Decomposition: Break the task into smaller steps. This helps the model reason through the task more effectively.

The Prompt

"You are a technical copywriter for developer tools. Write a value proposition for a browser-based video editing platform built for Ubuntu users.

Context: Built by a developer to solve the dual-boot problem. Uses WebAssembly and local storage. Features: timeline editing, cuts, trimming, MP4 export.

Follow these steps:

- Step 1: Identify the core user (who is this for?)
- Step 2: State the core problem (what frustrates them?)
- Step 3: Describe the solution (what does the product do?)

- Step 4: Explain how it works technically (why should they trust it?)
- Step 5: State the outcome (what do they get?)

Use this structure: Headline, Problem, Solution, Technical Trust, Result.
Keep the tone direct and plain—no buzzwords."

The Output

Headline: Edit video on Ubuntu. No NVIDIA. No rebooting.

Problem: You're a freelance video editor using Ubuntu. DaVinci Resolve won't run on your Intel GPU. Other Linux tools crash or lack basic features. Every client edit means rebooting into Windows—20 minutes wasted before you even start.

Solution: A browser-based editor built for Ubuntu. Timeline editing, cuts, trimming, MP4 export. No installation. No effects. No color grading. Just the essentials, done fast.

Technical Trust: WebAssembly powers near-native performance. IndexedDB stores everything locally—your footage never touches a server. No uploads. No privacy risks. No waiting.

Result: Turn around client cuts in minutes. Stay in Ubuntu. Ship work. Get paid. No compromises.

Iteration Notes

Note	Response
Technique applied	Step decomposition.
What changed in the output	Added a "Technical Trust" section that explicitly addresses credibility. The flow is logical: user → problem → solution → trust → outcome.
What improved	The output now builds trust systematically. The technical explanation (WebAssembly, IndexedDB) is separated from the solution description, making it clearer.

Note	Response
What still failed	The copy is good but could be more urgent and action-oriented in the opening.

VERSION 6: Final Best Prompt (Cleaned & Reusable)

This is the cleaned-up final prompt, incorporating all techniques. It works for a stranger without my personal context.

The Prompt

You are a technical copywriter specializing in developer tools and open-source software. Write a value proposition for a browser-based video editing platform.

Target Audience: Freelance video editors and content creators who use Ubuntu (or Linux) and need to do quick client cuts (trimming, splitting, MP4 export) without switching to Windows or relying on GPU-dependent software.

Product Details:

- Runs in Chrome (no installation)
- Uses WebAssembly for near-native performance
- Stores footage locally via IndexedDB (no server uploads)
- Features: timeline editing, cuts, trimming, MP4 export
- Does NOT support: effects, color grading, multi-track audio, 4K

The Problem It Solves: Ubuntu users with Intel (non-NVIDIA) GPUs cannot run DaVinci Resolve. Other Linux video editors are unstable or lack basic features. The only reliable option is dual-booting Windows, which adds friction and wasted time.

Follow these steps:

1. Identify the core user
2. State the core problem
3. Describe the solution (what it does)
4. Explain how it works technically (why they should trust it)
5. State the outcome (what they get)

Output Structure:

- **Headline** (one line)
- **Problem** (2-3 sentences)
- **Solution** (3-4 sentences)
- **Technical Trust** (2-3 sentences)
- **Result** (1-2 sentences)

Voice: Direct, plain, no buzzwords. Be honest about limitations. Sound like a developer explaining a tool to another developer—practical, credible, and slightly irreverent.

CROSS-MODEL COMPARISON: Claude vs. ChatGPT

Claude's Response (Final Prompt)

Headline: Edit video on Ubuntu. No NVIDIA. No rebooting.

Problem: You're a freelance video editor. You use Ubuntu. DaVinci Resolve won't run on your Intel GPU. Kdenlive crashes on export. Every client edit means rebooting into Windows—20 minutes wasted before you even open a file.

Solution: A browser-based video editor built for Ubuntu. Timeline editing, cuts, trimming, MP4 export. No installation. No effects. No color grading. Just the essentials, done reliably.

Technical Trust: WebAssembly delivers near-native performance. IndexedDB keeps everything on your machine—no uploads, no privacy risks, no waiting for server processing.

Result: Turn around client cuts in minutes. Stay in Ubuntu. Ship work. Get paid. No compromises.

ChatGPT's Response (Final Prompt)

Headline: Professional Video Editing on Ubuntu — No GPU Required

Problem: If you're a video editor using Ubuntu, you know the frustration. DaVinci Resolve demands an NVIDIA GPU, and other Linux tools often fall short. The result? You're forced to dual-boot Windows, losing time and momentum.

Solution: This browser-based video editor gives you a fast, reliable editing experience right in Chrome. Trim, cut, arrange on a timeline, and export MP4s — all without installations or GPU dependencies. It's built for speed and simplicity, focusing on the essentials you need for quick client work.

Technical Trust: Under the hood, WebAssembly ensures smooth performance, while IndexedDB keeps your footage local — nothing is uploaded to the cloud. Your data stays private, and you stay in control.

Result: Finish edits faster. Skip the reboot. Deliver quality work — all from your Ubuntu machine.

Comparison Notes

Dimension Claude

Tone

More direct and plain. "No compromises" hits harder.

Structure

Followed the requested structure exactly (Headline → Problem → Solution → Technical Trust → Result).

ChatGPT

Slightly more polished and "professional" in a corporate sense.

Followed the structure but added more transitional phrasing ("If you're...", "The result?").

Dimension	Claude	ChatGPT
Specificity	Mentioned "Kdenlive crashes on export" — a concrete detail.	More general ("other Linux tools often fall short").
Voice	Sounds like a developer talking to a developer.	Sounds like a marketer talking to a developer.
Weakness	The "Technical Trust" section is slightly dry.	The "Technical Trust" section is more explanatory but less punchy.
Verdict	Claude's version is tighter and more authentic to the intended voice. ChatGPT's is slightly more polished but loses some of the raw, developer-to-developer credibility.	

Specific observation: Claude used "Kdenlive crashes on export" — a concrete, verifiable detail that signals deep knowledge of the Linux ecosystem. ChatGPT used the vaguer "other Linux tools often fall short." This is a meaningful difference in accuracy and specificity.

FINAL REUSABLE TEMPLATE

markdown

You are a [ROLE] specializing in [DOMAIN]. Write a [TYPE OF CONTENT] for [PRODUCT/SERVICE].

****Target Audience:**** [Specific user persona — not "everyone"]

****Product Details:****

- [Feature 1]
- [Feature 2]
- [Feature 3]
- Does NOT support: [Limitations — be honest]

****The Problem It Solves:**** [What frustrates the user? What's broken?]

****Follow these steps:****

1. [Step 1]
2. [Step 2]
3. [Step 3]

- 4. [Step 4]
- 5. [Step 5]

****Output Structure:****

- ****Headline**** (one line)
- ****Problem**** (2-3 sentences)
- ****Solution**** (3-4 sentences)
- ****[Optional Section]**** (2-3 sentences)
- ****Result**** (1-2 sentences)

****Voice:**** [Tone descriptor, e.g., "Direct, plain, no buzzwords. Sound like a practitioner explaining to another practitioner."]

SUMMARY: Improvements Across Versions

Version	Technique	Key Improvement
0	None	Generic, unusable.
1	Role Assignment	Tone shifted to technical and direct.
2	Context & Motivation	Added personal story and "why."
3	Few-Shot Examples	Structure became cleaner, benefit-focused.
4	Output Structure	Scannable, professional format.
5	Step Decomposition	Added credibility section, logical flow.
6	Final Cleaned Prompt	Reusable by anyone.